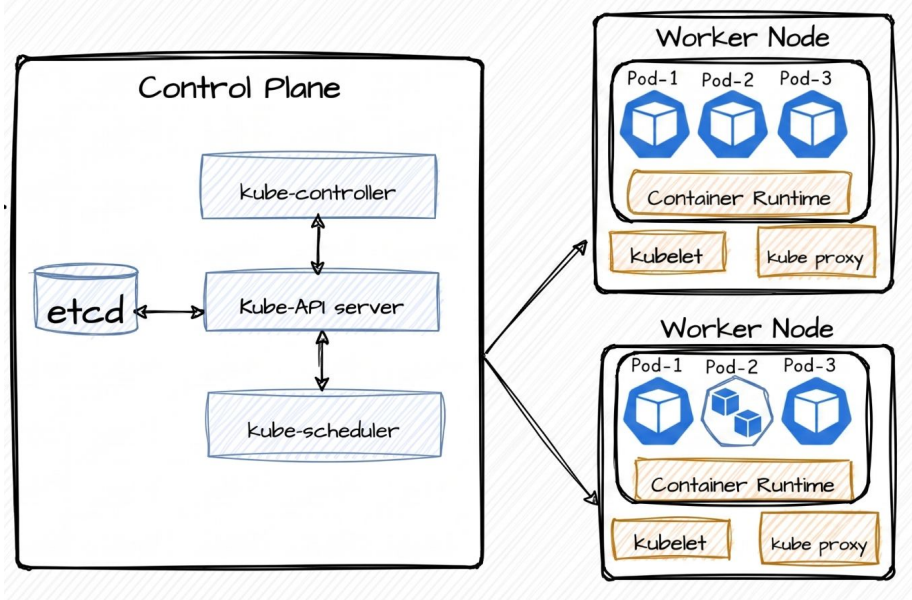
Gerência de configuração

Prática de Kubernetes

Prof. Diego Cardoso Borda Castro

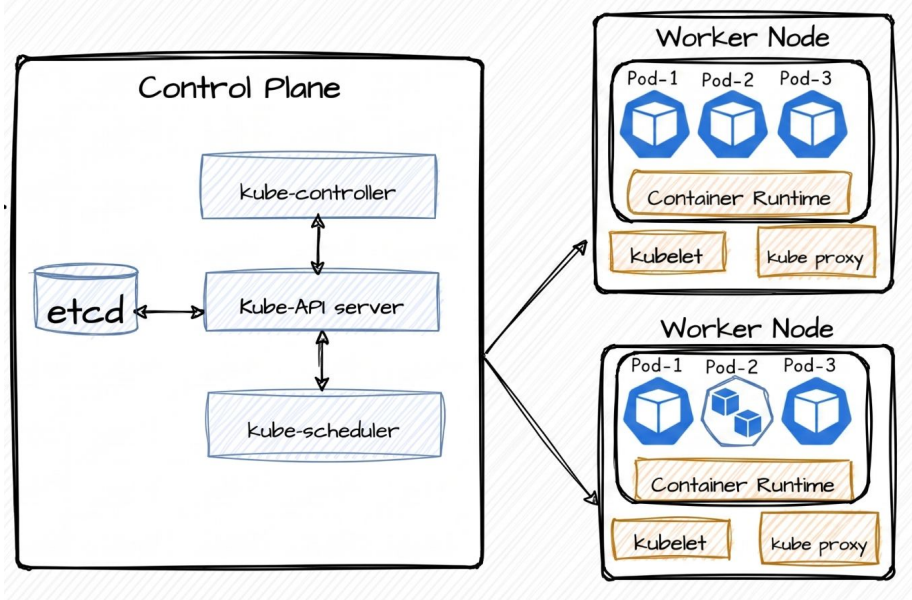
Control plane

- **API Server**
  - Porta de entrada do cluster: o kubectl só conversa com ele
- **etcd**
  - Banco chave/valor com o estado do cluster
- **Scheduler**
  - Escolhe em qual node cada Pod vai rodar
- **Controller Manager**
  - Compara desejado x atual e corrige (Deployment, ReplicaSet, Node, Job…)

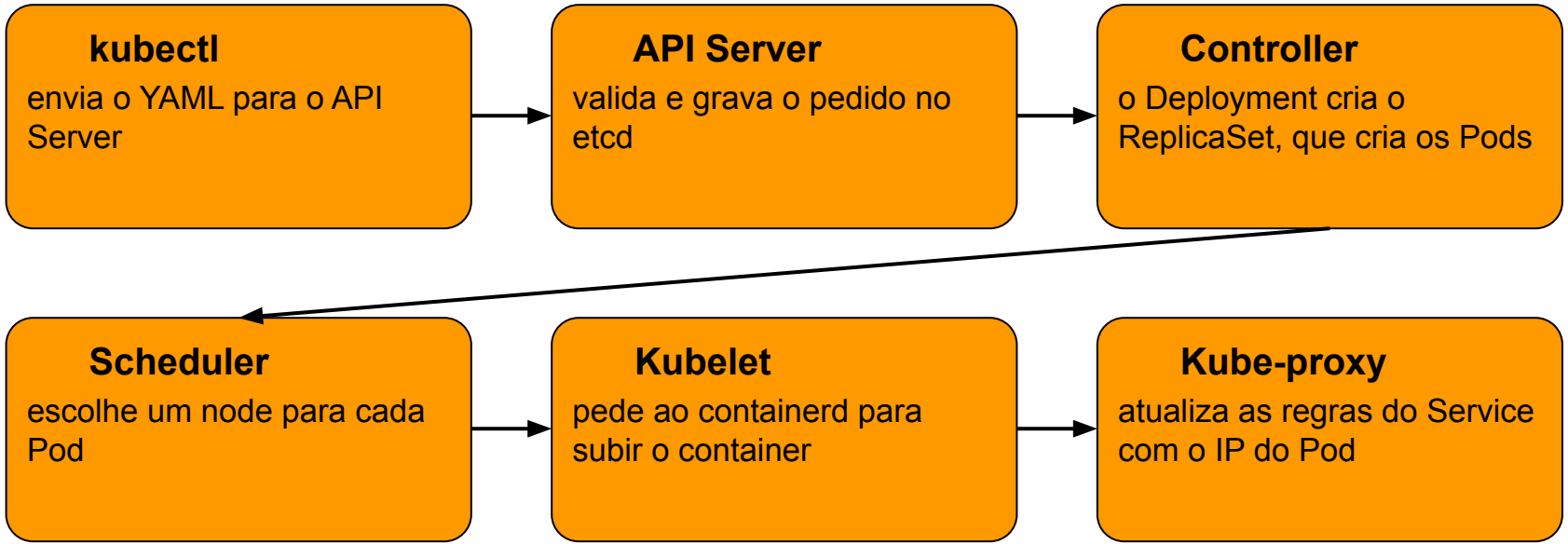


Worker nodes

- **Kubelet**
  - Agente em cada node. Recebe ordens do API Server e garante que os Pods estão rodando
- **Kube-proxy**
  - Cria as regras de rede que fazem o Service funcionar e balancear
- **Container runtime**
  - containerd: quem realmente cria e roda os containers
- No k3d, cada node é um container Docker



O que faz o kubectl apply?



Objetos que vamos usar

Deployment
Mantém N réplicas e faz rollout/rollback

ReplicaSet
Criado pelo Deployment. Conta os Pods

Service
IP e DNS fixos + balanceamento entre Pods

Ingress
Entrada HTTP do cluster (Traefik no k3d)

ConfigMap
Configurações em chave/valor

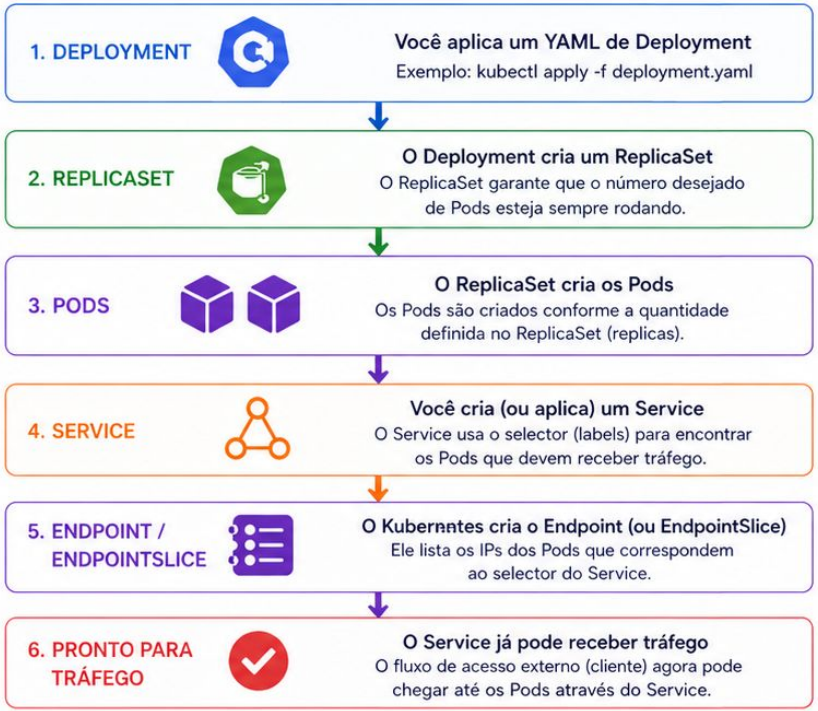
Secret
Dados sensíveis (senhas, tokens)

StatefulSet
Pods com nome fixo e disco próprio (banco)

HPA
Escala os Pods sozinho pela CPU

Estrutura hierárquica

- Deployment → ReplicaSet → Pods
  - Você só escreve o Deployment
- Service encontra os Pods pelo label
  - Não importa o IP nem o node
- Ingress → Service
  - Regras HTTP para dentro do cluster



Labels e selectors

deployment.yaml

```yaml
spec:
  selector:
    matchLabels:
      app: api-js
  template:
    metadata:
      labels:
        app: api-js
```

service.yaml

```yaml
spec:
  selector:
    app: api-js
  ports:
    - port: 80
      targetPort: 3000
```



Label é uma etiqueta chave/valor. Selector é a busca por etiqueta.
Label errado = Service sem Pods → confira com `kubectl get endpoints api-js`

Tipos de Service

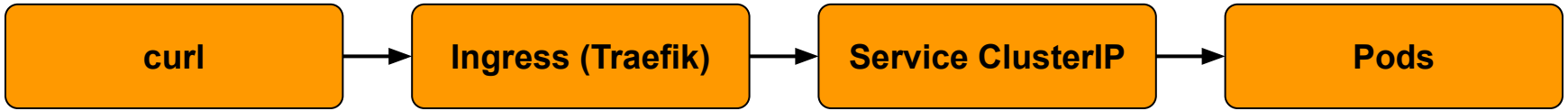
ClusterIP
Padrão. Só acessível dentro do cluster

NodePort
Abre uma porta 30000-32767 em todos os nodes

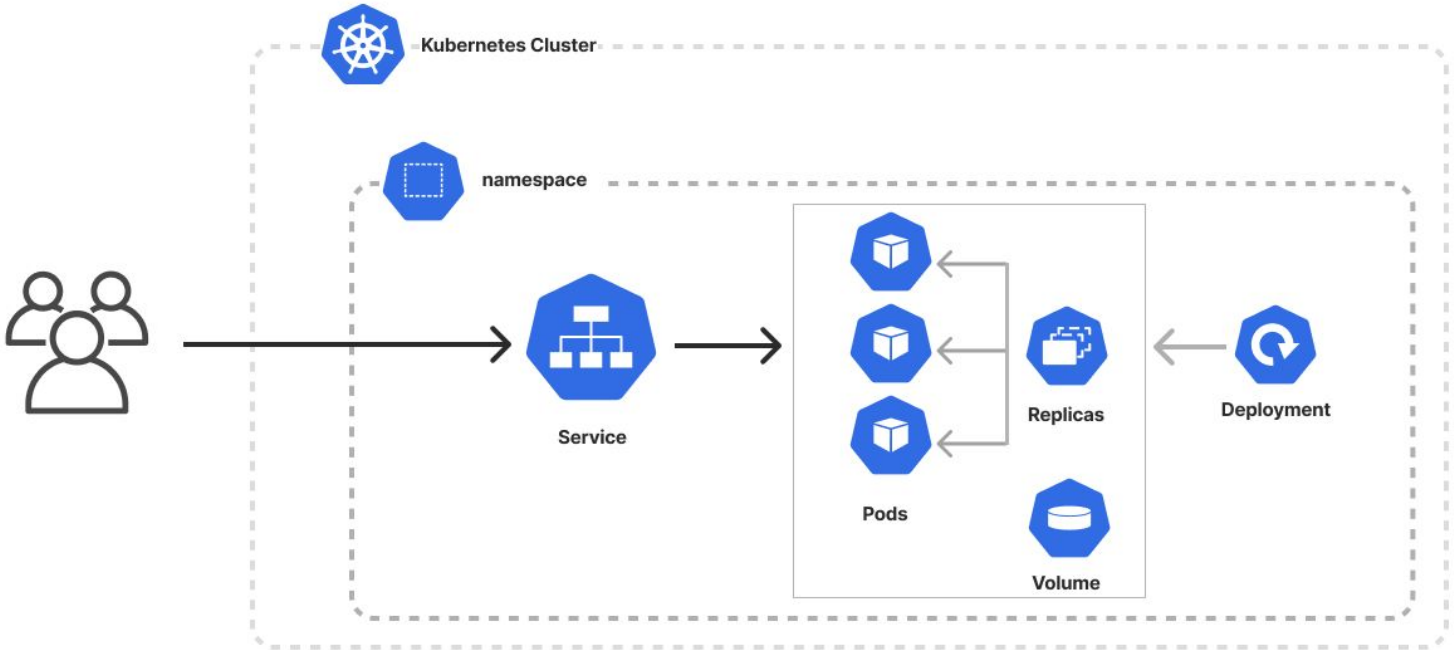
LoadBalancer
Pede um IP externo (nuvem, MetalLB)

ExternalName
Apelido DNS para um serviço de fora

Caminho de uma requisição nos nossos exemplos:



Caminho da requisição



Formas de escalar

Vertical

- Mais CPU e memória para o mesmo Pod
- resources: requests / limits
- Limite: o tamanho do node

Horizontal: Pods

- Mais réplicas da aplicação
- Manual: kubectl scale ou replicas no YAML
- Automático: HPA

Horizontal: Nodes

- Mais máquinas no cluster
- Local: k3d node create
- Nuvem: Cluster Autoscaler, Karpenter

Hoje: **Pods na mão** (exemplo 1), **Pods automático com HPA** (exemplo 2) e **Nodes** nos dois exemplos

Comandos do dia a dia

kubectl <verbo> <recurso> [nome]

`kubectl get pods -o wide`	lista os Pods e o node de cada um
`kubectl get pods -w`	fica observando as mudanças
`kubectl describe pod <nome>`	detalhes e eventos (ótimo para erro)
`kubectl logs <pod>`	logs do container
`kubectl apply -f ./k8s`	cria ou atualiza a partir dos YAML
`kubectl delete -f ./k8s`	remove o que os YAML criaram
`kubectl scale deployment api-js --replicas=5`	muda o número de réplicas

Ferramentas

Docker
O k3d roda o k3s (Kubernetes leve) dentro de containers

kubectl
Cliente de linha de comando que fala com o API Server

k3d
Cria e apaga clusters locais em segundos

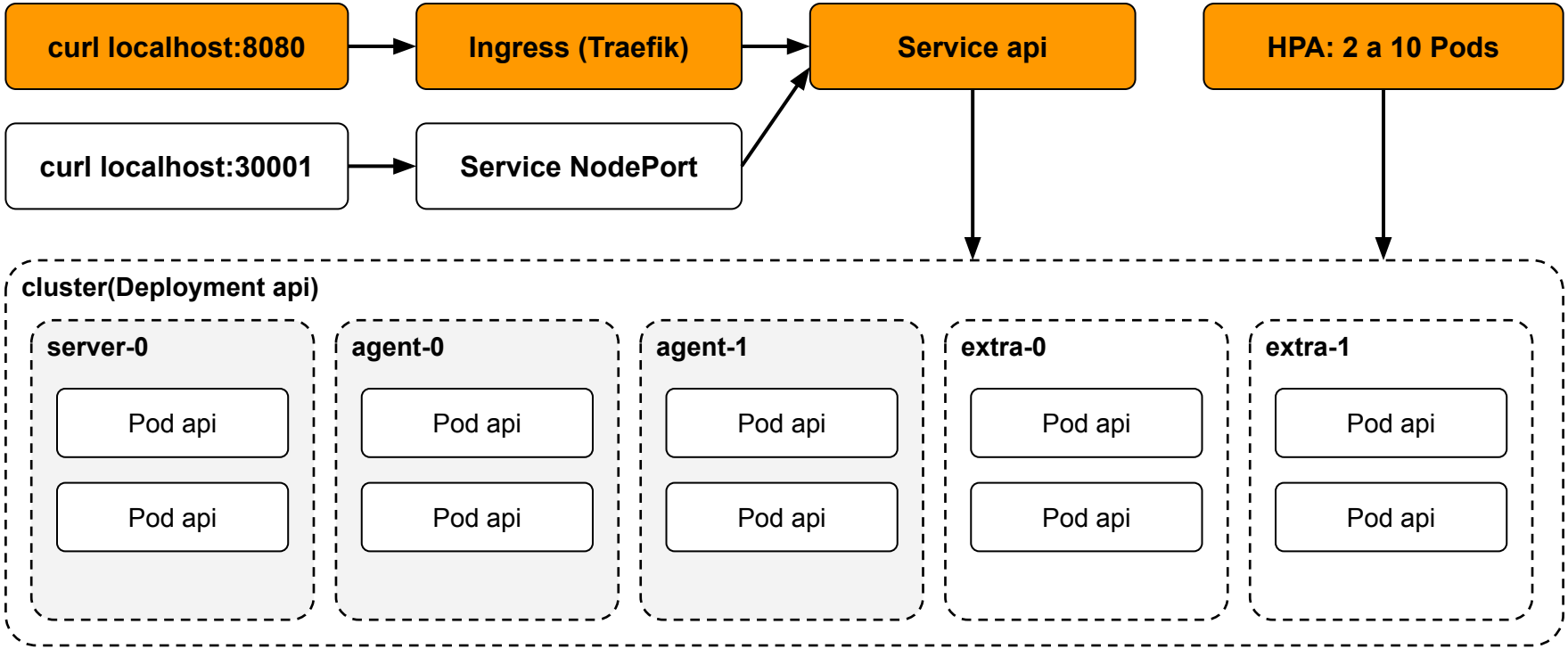
kubectl

```
$ curl -LO https://dl.k8s.io/release/v1.31.0/bin/linux/amd64/kubectl
$ sudo install kubectl /usr/local/bin/kubectl
```

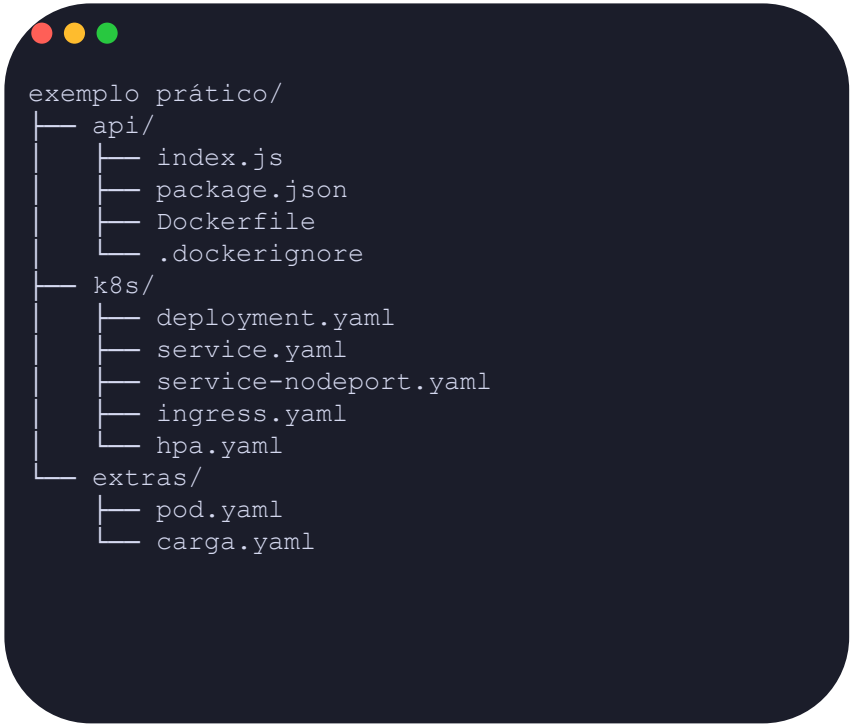
k3d

```
$ curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
$ k3d version
```

O que vamos construir



Passo 1: as pastas



- **api/**: o código e a imagem
- **k8s/**: o que fica rodando no cluster
- **extras/**: o que usamos só para testar
  - pod.yaml: um Pod sozinho
  - carga.yaml: gera carga para o HPA

Passo 2: a API

api/index.js

```
import express from 'express';
import os from 'os';

const app = express();
const port = 3000;

app.get('/info', (req, res) => {
    res.json({ pod: os.hostname(), node: process.env.NODE_NAME ||'local' });
});

app.get('/cpu', (req, res) => {
    const inicio = Date.now();
    while (Date.now() - inicio <50) Math.sqrt(Math.random());
    res.json({ pod: os.hostname() });
});

app.listen(port, () => {
    console.log(`Rodando na porta ${port}.);
});

process.on('SIGTERM', () => process.exit(0));
```

- **/info**
  - Nome do Pod e do node que respondeu
- **/cpu**
  - Gasta 50ms de CPU: vai acionar o HPA
- **SIGTERM**
  - O Kubernetes avisa antes de matar o Pod. Sem isso ele espera 30s

Passo 3: Dockerfile

api/package.json

```
{
  "name": "api-k8s",
  "version": "1.0.0",
  "type": "module",
  "main": "index.js",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

api/Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

api/.dockerignore

```
node_modules
```

- Só uma dependência: express
- package.json antes do código: cache do npm install

Passo 4: build e teste local

```
$ cd "kubernetes/exemplo prático"
$ docker build -t api:1.0 ./api
$ docker run --rm -p 3000:3000 api:1.0
Rodando na porta 3000.
```

```
        outro terminal

$ curl localhost:3000/info
{"pod":"e27313faa1a0","node":"local"}
```

- pod = ID do container
- node = local: ainda não tem Kubernetes

Passo 5: criar o cluster

```
k3d cluster create aula9 --servers 1 --agents 2 \
-p "8080:80@loadbalancer" -p "30001:30001@loadbalancer"
```

- **--servers 1**: um node de control plane
- **--agents 2**: dois worker nodes
- **-p "8080:80@loadbalancer"**: porta 8080 da máquina → Ingress
- **-p "30001:30001@loadbalancer"**: porta 30001 da máquina → NodePort

Passo 6: conferir e importar

```
$ kubectl get nodes
NAME                   STATUS   ROLES
k3d-aula9-agent-0      Ready    <none>
k3d-aula9-agent-1      Ready    <none>
k3d-aula9-server-0     Ready    control-plane,master

$ docker ps --format "{{.Names}}"
k3d-aula9-serverlb
k3d-aula9-agent-1
k3d-aula9-agent-0
k3d-aula9-server-0
```

```
$ k3d image import api:1.0 -c aula9
```

- Cada node é um container Docker
- serverlb: o load balancer na frente do cluster
- Os nodes não enxergam as imagens do seu Docker

Passo 7: criando um Pod

extras/pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: api-pelado
  labels:
    app: api
spec:
  containers:
    - name: api
      image: api:1.0
      imagePullPolicy: IfNotPresent
      ports:
        - containerPort: 3000
```

```
$ kubectl apply -f extras/pod.yaml
pod/api-pelado created
$ kubectl get pods -o wide
NAME          READY   STATUS    NODE
api-pelado    1/1     Running   k3d-aula9-agent-1
```

- **kind: Pod**: a menor unidade
- **imagePullPolicy: IfNotPresent**: usa a imagem importada
- O scheduler escolheu o node

Passo 8: acessar o Pod

```
$ kubectl port-forward pod/api-pelado 3000:3000
$ curl localhost:3000/info
{"pod":"api-pelado","node":"local"}
```

```
$ kubectl logs api-pelado
Rodando na porta 3000.
```

```
$ kubectl delete pod api-pelado
$ kubectl get pods
No resources found
```

- **port-forward**: liga uma porta da sua máquina direto no Pod
- Apagou o Pod, ninguém cria outro. Por isso usamos **Deployment**

Passo 9: Deployment (1/2)

k8s/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
```

- **replicas: 3**: quantas cópias do Pod
- **selector.matchLabels**: quais Pods são deste Deployment
- **template**: a "forma" de cada Pod
- O label do template precisa ser igual ao do selector

Passo 9: Deployment (2/2)

k8s/deployment.yaml (continuação)

```yaml
spec:
  containers:
    - name: api
      image: api:1.0
      imagePullPolicy: IfNotPresent
      ports:
        - containerPort: 3000
      env:
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
      resources:
        requests:
          cpu: "100m"
        limits:
          cpu: "300m"
```

- **containers**: igual ao Pod do passo 7
- **env NODE_NAME com fieldRef**
  - O Kubernetes coloca o nome do node na variável
- **resources**
  - requests: o mínimo reservado. O scheduler e o HPA usam
  - limits: o máximo que o container pode usar

Passo 10: o que foi criado

```
$ kubectl apply -f k8s/deployment.yaml
$ kubectl get deployment,replicaset,pods
NAME                  READY   UP-TO-DATE   AVAILABLE
deployment.apps/api   3/3     3            3

NAME                                DESIRED   CURRENT   READY
replicaset.apps/api-7875d9d9bc      3         3         3

NAME                          READY   STATUS
pod/api-7875d9d9bc-8g765      1/1     Running
pod/api-7875d9d9bc-khvqj      1/1     Running
pod/api-7875d9d9bc-pplq9      1/1     Running
```

- Nós criamos 1 objeto. O Kubernetes criou o ReplicaSet e os 3 Pods
- O nome do Pod = nome do ReplicaSet + sufixo

Passo 11: labels na prática

```
$ kubectl get pods --show-labels
NAME                   READY   STATUS    LABELS
api-7875d9d9bc-8g765   1/1     Running   app=api,pod-template-hash=7875d9d9bc
api-7875d9d9bc-khvqj   1/1     Running   app=api,pod-template-hash=7875d9d9bc
api-7875d9d9bc-pplq9   1/1     Running   app=api,pod-template-hash=7875d9d9bc

$ kubectl get pods -l app=api -o wide
NAME                   READY   STATUS    IP          NODE
api-7875d9d9bc-8g765   1/1     Running   10.42.1.6   k3d-aula9-agent-1
api-7875d9d9bc-khvqj   1/1     Running   10.42.2.5   k3d-aula9-server-0
api-7875d9d9bc-pplq9   1/1     Running   10.42.0.5   k3d-aula9-agent-0
```

- **-l app=api** filtra por label: é a mesma busca que o Service vai fazer
- O scheduler espalhou um Pod em cada node

Passo 12: Service ClusterIP

k8s/service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 3000
```

```
$ kubectl apply -f k8s/service.yaml
$ kubectl get svc api
NAME   TYPE        CLUSTER-IP
api    ClusterIP   10.43.71.231

$ kubectl get endpoints api
api   10.42.0.5:3000,10.42.1.6:3000,
      10.42.2.5:3000
```

- Sem **type** = ClusterIP: IP fixo, só dentro do cluster. Porta 80 → 3000 do Pod
- **endpoints**: os IPs dos Pods que o selector encontrou

Passo 13: teste interno

```
$ kubectl run teste --rm -it --restart=Never --image=busybox:1.36 -- wget -qO- http://api/info
{"pod":"api-7875d9d9bc-khvqj","node":"k3d-aula9-server-0"}
pod "teste" deleted
```

- **kubectl run**: cria um Pod temporário só para o teste
- **--rm**: apaga o Pod quando o comando termina
- **http://api**: o nome do Service vira um DNS dentro do cluster
- De fora (seu navegador) o ClusterIP não funciona. Para isso: NodePort ou Ingress

Passo 14: Service NodePort

k8s/service-nodeport.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api-nodeport
spec:
  type: NodePort
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30001
```

```
$ kubectl apply -f k8s/service-nodeport.yaml
$ kubectl get svc
NAME           TYPE        PORT(S)
api            ClusterIP   80/TCP
api-nodeport   NodePort    80:30001/TCP
kubernetes     ClusterIP   443/TCP
```

```
$ curl localhost:30001/info
{"pod":"api-7875d9d9bc-khvqj",
 "node":"k3d-aula9-server-0"}
```

- Abre a porta 30001 em todos os nodes (30000-32767)

Passo 15: Ingress

k8s/ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
```

```
$ kubectl apply -f k8s/ingress.yaml
$ kubectl get ingress
NAME    CLASS     HOSTS   PORTS
api     traefik   *       80
```

- O k3d já vem com o **Traefik**
- Sem **host**: aceita qualquer endereço
- **path: /**: tudo vai para o Service api
- Deu 404? O Traefik leva ~1 min para subir: `kubectl get pods -n kube-system`

Passo 16: balanceamento

```
$ for i in $(seq 1 6); do curl -s localhost:8080/info; echo; done
{"pod":"api-7875d9d9bc-khvqj","node":"k3d-aula9-server-0"}
{"pod":"api-7875d9d9bc-pplq9","node":"k3d-aula9-agent-0"}
{"pod":"api-7875d9d9bc-8g765","node":"k3d-aula9-agent-1"}
{"pod":"api-7875d9d9bc-khvqj","node":"k3d-aula9-server-0"}
{"pod":"api-7875d9d9bc-pplq9","node":"k3d-aula9-agent-0"}
{"pod":"api-7875d9d9bc-8g765","node":"k3d-aula9-agent-1"}
```

localhost:8080 → load balancer do k3d → Ingress → Service → Pods
Cada requisição cai em um Pod (e node) diferente

Passo 17: Pods subindo

terminal 1: observar

```
$ kubectl get pods -o wide -w
```

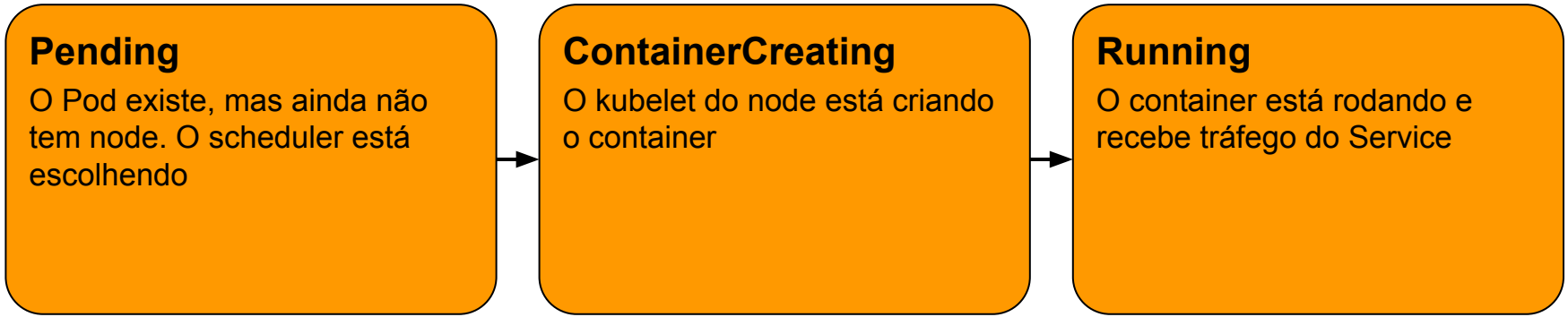
terminal 2: escalar

```
$ kubectl scale deployment api --replicas=6
```

terminal 1

```
NAME                   READY   STATUS              NODE
api-7875d9d9bc-r5kfz   0/1     Pending             <none>
api-7875d9d9bc-86ljq   0/1     Pending             <none>
api-7875d9d9bc-j27ds   0/1     Pending             <none>
api-7875d9d9bc-j27ds   0/1     Pending             k3d-aula9-server-0
api-7875d9d9bc-r5kfz   0/1     ContainerCreating   k3d-aula9-agent-0
api-7875d9d9bc-86ljq   0/1     ContainerCreating   k3d-aula9-agent-1
api-7875d9d9bc-j27ds   1/1     Running             k3d-aula9-server-0
api-7875d9d9bc-r5kfz   1/1     Running             k3d-aula9-agent-0
api-7875d9d9bc-86ljq   1/1     Running             k3d-aula9-agent-1
```

As fases do Pod



- De 3 para 6 Pods em poucos segundos
- Os 3 novos foram espalhados, um em cada node
- `kubectl get endpoints api` agora mostra 6 IPs

Passo 18: Pods descendo

```
$ kubectl apply -f k8s/deployment.yaml
deployment.apps/api configured
```

terminal 1

```
NAME                   READY   STATUS        AGE
api-7875d9d9bc-86ljq   1/1     Terminating   9s
api-7875d9d9bc-j27ds   1/1     Terminating   9s
api-7875d9d9bc-r5kfz   1/1     Terminating   9s
api-7875d9d9bc-j27ds   0/1     Completed     9s
```

- O arquivo diz **replicas: 3**: o apply desfaz o scale. O arquivo no Git manda
- Terminating: recebeu o SIGTERM e está saindo

Passo 19: self healing

```
$ kubectl delete pod api-7875d9d9bc-8g765
```

terminal 1

```
NAME                   READY   STATUS              AGE
api-7875d9d9bc-8g765   1/1     Terminating         42s
api-7875d9d9bc-q7bjt   0/1     Pending             0s
api-7875d9d9bc-q7bjt   0/1     ContainerCreating   0s
api-7875d9d9bc-q7bjt   1/1     Running             2s
```

- O ReplicaSet viu 2 de 3 e criou outro em 2 segundos
- Nome e IP novos: quem usa o Service nem percebe

Passo 20: investigando um Pod

```
$ kubectl exec api-7875d9d9bc-khvqj -- sh -c 'hostname; echo $NODE_NAME'
api-7875d9d9bc-khvqj
k3d-aula9-server-0
```

```
$ kubectl describe pod api-7875d9d9bc-khvqj
Events:
  Normal  Scheduled  Successfully assigned default/api-7875d9d9bc-khvqj to k3d-aula9-server-0
  Normal  Started    Started container api
```

- **exec**: roda um comando dentro do container (-it sh para entrar)
- **describe**: os eventos. **logs**: o que a aplicação imprimiu

Passo 21: rollout para a 2.0

```
$ docker tag api:1.0 api:2.0
$ k3d image import api:2.0 -c aula9
$ kubectl set image deployment/api api=api:2.0
$ kubectl annotate deployment/api kubernetes.io/change-cause="versão 2.0"
```

terminal 1 (resumido)

```
api-665dd77c6-zb6nj    1/1   Running
api-7875d9d9bc-pplq9   1/1   Terminating
api-665dd77c6-nv8bg    1/1   Running
api-7875d9d9bc-q7bjt   1/1   Terminating
api-665dd77c6-m687t    1/1   Running
api-7875d9d9bc-khvqj   1/1   Terminating
```

- Rolling update: sobe um novo, derruba um antigo. A API nunca fica fora do ar

Passo 22: histórico e rollback

```
$ kubectl get rs
NAME             DESIRED   READY
api-665dd77c6    3         3
api-7875d9d9bc   0         0

$ kubectl rollout history deployment/api
REVISION  CHANGE-CAUSE
1         <none>
2         versão 2.0
```

```
$ kubectl rollout undo deployment/api
deployment.apps/api rolled back

$ kubectl get deployment api -o wide
NAME   READY   IMAGES
api    3/3     api:1.0
```

- Cada versão ganha um ReplicaSet. O antigo fica com 0 Pods, pronto para voltar
- **rollout undo**: volta para a versão anterior com o mesmo rolling update
- `kubectl rollout status deployment/api` acompanha o andamento

Passo 23: mais nodes

```
$ k3d node create extra --cluster aula9 --role agent --replicas 2
$ k3d image import api:1.0 -c aula9
```

```
$ kubectl get nodes
NAME                 STATUS   ROLES
k3d-aula9-agent-0    Ready    <none>
k3d-aula9-agent-1    Ready    <none>
k3d-aula9-server-0   Ready    control-plane,master
k3d-extra-0          Ready    <none>
k3d-extra-1          Ready    <none>
```

- De 3 para 5 nodes
- Importe de novo: node novo não tem a imagem
  - Sem isso: ErrImagePull
- Os Pods atuais não mudam de node sozinhos

Passo 24: HPA

k8s/hpa.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 30
```

- Escala o Deployment **api** entre **2 e 10** Pods
- Meta: CPU média = **50%** do request (50m de 100m)
- **scaleDown 30s**: o padrão é 5 min

```
$ kubectl apply -f k8s/hpa.yaml
$ kubectl get hpa
NAME   TARGETS       MINPODS   MAXPODS   REPLICAS
api    cpu: 1%/50%   2         10        3
```

Passo 25: gerador de carga

extras/carga.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: carga
spec:
  replicas: 2
  selector:
    matchLabels:
      app: carga
  template:
    metadata:
      labels:
        app: carga
    spec:
      containers:
        - name: carga
          image: busybox:1.36
          command: ["sh", "-c", "while true; do wget -q -O- http://api/cpu; done"]
```

- busybox chamando /cpu sem parar
- Usa o DNS do Service: http://api
- 2 réplicas: dá para escalar a carga também

```
$ kubectl get hpa -w
```

```
$ kubectl get pods -o wide -w
```

```
$ kubectl apply -f extras/carga.yaml
```

Passo 26: HPA subindo

kubectl get hpa -w

```
NAME   TARGETS         REPLICAS   AGE
api    cpu: 1%/50%     3          15s
api    cpu: 21%/50%    3          30s
api    cpu: 182%/50%   3          45s
api    cpu: 163%/50%   7          60s
api    cpu: 87%/50%    10         75s
api    cpu: 75%/50%    10         90s
```

3 → 7 → 10 Pods em ~30 segundos
182% com 3 Pods → ⌈3 × 182 ÷ 50⌉ = 11
limitado ao máximo: 10

10 Pods em 5 nodes

```
NAME                    NODE
api-7875d9d9bc-v7rtg    k3d-aula9-agent-0
api-7875d9d9bc-z7cmk    k3d-aula9-agent-0
api-7875d9d9bc-8hzgn    k3d-aula9-agent-1
api-7875d9d9bc-ntxz9    k3d-aula9-agent-1
api-7875d9d9bc-gkx8v    k3d-aula9-server-0
api-7875d9d9bc-ts6rv    k3d-aula9-server-0
api-7875d9d9bc-h4vn8    k3d-extra-0
api-7875d9d9bc-mv22w    k3d-extra-0
api-7875d9d9bc-8jgrb    k3d-extra-1
api-7875d9d9bc-mh4tj    k3d-extra-1
```

Passo 27: HPA descendo

```
$ kubectl delete -f extras/carga.yaml
```

```
kubectl get hpa -w

NAME   TARGETS         REPLICAS   AGE
api    cpu: 62%/50%    10         2m45s
api    cpu: 7%/50%     10         3m
api    cpu: 3%/50%     10         3m15s
api    cpu: 1%/50%     2          3m30s
api    cpu: 0%/50%     2          4m
```

- Sem carga a CPU cai
- Espera a janela de 30s
- **10 → 2 Pods**: volta ao minReplicas
- 8 Pods em Terminating ao mesmo tempo

Passo 28: tirar os nodes extras

```
$ kubectl drain k3d-extra-0 --ignore-daemonsets --delete-emptydir-data
$ k3d node delete k3d-extra-0
$ kubectl delete node k3d-extra-0
```

- **drain**: não aceita Pods novos e tira os que estão lá com segurança
- **k3d node delete**: apaga o container do node
- **kubectl delete node**: tira o node da lista do cluster
  - Sem isso ele fica aparecendo como SchedulingDisabled
- Repita para o k3d-extra-1

Resumo do exemplo prático

Pod	`kubectl apply -f extras/pod.yaml`	não se recupera
Deployment	`kubectl apply -f k8s/deployment.yaml`	cria ReplicaSet e Pods
Services	`ClusterIP, NodePort :30001`	IP fixo + balanceamento
Ingress	`localhost:8080`	entrada HTTP
Pods ↑↓ na mão	`kubectl scale / kubectl apply`	3 → 6 → 3
Rollout	`set image / rollout undo`	1.0 → 2.0 → 1.0
Pods ↑↓ sozinhos	`HPA + carga`	3 → 10 → 2
Nodes ↑↓	`k3d node create / drain + delete`	3 → 5 → 3